

```
// Code-A4 (0-1 Knapsack)
```

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
int knapsack(int W, vector<int>& wt, vector<int>& val, int n) {
```

```
    // dp[i][w] = max value for first i items with weight limit w
```

```
    vector<vector<int>> dp(n + 1, vector<int>(W + 1, 0));
```

```
    // Build the table dp[][] bottom-up
```

```
    for (int i = 1; i <= n; i++) {
```

```
        for (int w = 1; w <= W; w++) {
```

```
            if (wt[i - 1] <= w)
```

```
                dp[i][w] = max(val[i - 1] + dp[i - 1][w - wt[i - 1]], dp[i - 1][w]);
```

```
            else
```

```
                dp[i][w] = dp[i - 1][w];
```

```
        }
```

```
    }
```

```
    return dp[n][W]; // The bottom-right cell gives the result
```

```
}
```

```
int main() {
```

```
    int n, W;
```

```
    cout << "Enter number of items: ";
```

```
    cin >> n;
```

```
    vector<int> val(n), wt(n);
```

```
    cout << "Enter values of items:\n";
```

```
    for (int i = 0; i < n; i++) cin >> val[i];
```

```
cout << "Enter weights of items:\n";
for (int i = 0; i < n; i++) cin >> wt[i];

cout << "Enter capacity of knapsack: ";
cin >> W;

int maxValue = knapsack(W, wt, val, n);
cout << "\nMaximum value in knapsack = " << maxValue << endl;

return 0;
}
```

```
// SAMPLE OUTPUT
```

```
/*
```

```
* $ ./a.out
```

```
* Enter number of items: 3
```

```
* Enter values of items:
```

```
* 12
```

```
* 32
```

```
* 14
```

```
* Enter weights of items:
```

```
* 45
```

```
* 34
```

```
* 65
```

```
* Enter capacity of knapsack: 60
```

```
*
```

```
* Maximum value in knapsack = 32
```

```
*/
```